

Incident Report: Trade Duplication (3–75× per signal) — claim check bypassed by message-revision path

Date: 2026-08-04 (investigation); duplication observed since 2026-07-23

Severity: Critical

- Real broker orders were placed 3–75× per signal on live/demo accounts.
- User Luis ESP reported that trades were duplicating.

Evidence:

- `docs/Prod_Logs/Trade/logs.1785865057109.log` — 24,999 lines, 2026-08-04 14:24–15:08 UTC.
- `docs/Prod_Logs/Listener/logs.1785803111851.log` — 2026-08-01 07:07–10:42 UTC.
- Live production DB `sxkpcovbyaficvtpksdo` — `trades`, `signals`, `signal_broker_dispatch_claims`, `listener_events`, and `broker_accounts`.

Root cause:

- The only anti-duplicate guard, `signal_broker_dispatch_claims`, is skipped on the message-revision execution path.
- The listener re-fetches Telegram messages during the entry settle poll. A text difference is treated as a revision and re-dispatched with `dispatch_source=message_revision`.
- `TradeExecutor.ts:1466` sees `isRevisionRefresh` and skips the claim check, so `OrderSend` runs again.
- The amend-only merge path is conditional. When it returns `handled: false`, execution falls through to the full entry plan and opens duplicate orders.
- Repeated settle polls, reconciliations, catch-up processing, and live edits can repeat the cycle, producing 3–75 identical positions on one account.

Root cause in plain English:

- The system has a safety check that should allow one signal to open one set of trades per broker account.
- After opening the trade, the listener checks Telegram again a few seconds later.
- When that second check sees a changed message, the system treats it as a new instruction instead of an update to the existing trade.
- The code then skips the safety check and sends the same order again.
- Repeated message checks repeat the same mistake, creating several real broker orders from one signal.
- The correct behavior is to update the existing trade's stop-loss or take-profit, or do nothing when no update is needed. It must not open the trade again.

Important nuance — verified 2026-08-05:

- Both confirmed duplicate signals had real Telegram edits: `906a4b64` at 11:52:40 and `ead1ebb8` at 13:41:51.
- Requiring a real edit date is therefore not sufficient.
- Every revision must amend the open basket or skip; it must never re-enter, regardless of edit date.

Secondary bug:

- Some duplicated signals, including `906a4b64`, have `channel_id = NULL`, which bypasses `enforce_signal_channel_filter`.
- The NULL-channel bypass affected the 44Fx group. It did not affect `ead1ebb8`, whose channel was legitimately allowed for the FTMO account.

Status:

- Worker fix implemented in source and verified locally; deployment and staging verification are still pending.
- The database defense-in-depth migration is not yet implemented, so full system idempotency is not complete until that migration and the worker fix are deployed.

Related incidents:

- `docs/incident-2026-07-28-telegram-reconnect-storm.md`
- `docs/incident-2026-07-31-listener-crash-loop.md`

1. Scope & Impact

Metric	Value
Affected users (14 days)	10+ (trades-per-signal ≥ 2×); worst: 14bf6329 (1,498 excess), Luis dd18ad68 (1,352 excess) , d7b3e671 (559), a23f24d9 (521), 3f8a2ff4 (276)
Luis — signals with duplicates	56 of 81 signals (14 days), 1,408 trades in duplicated groups (~86 expected → ~1,300 excess)
Worst single signal	msg #17219 → 75 identical orders (Aug 3 08:50:25–08:55:43 UTC)
Luis's worst day	Aug 4: 8 duplicated signal groups → 212 trades (53, 36, 34, 30, 20, 19, 17, 3 — the 3 on FTMO, closed together Aug 5 00:19:33 ; groups >20 also exceed his <code>multi_trade_max_orders: 20</code> cap)
Per-trade evidence	34 distinct broker tickets (1839647592→1839649459) for ONE signal on ONE account, one every ~0.37s
Duplication start	At least 2026-07-23 (msg #2922 → 20 trades)
Resolved (positions)	Signal <code>ead1ebb8</code> (44's Club, msg #14238): 3× 0.41-lot XAUUSD buys on FTMO USD 100K — all 3 closed 2026-08-05 00:19:33 (no longer open); signal <code>906a4b64</code> 34× — all closed

2. What Happened (Plain English)

A Telegram signal arrives: "open ONE XAUUSD trade". The system opens it **many times**. Copies appear ~0.37 seconds apart, with identical symbol, lot, SL and TP, and are closed together.

Why: the system has a safety bookmark ("this signal is already dispatched for this broker"). But ~10 seconds after the first order, the listener polls Telegram again

to see if the channel edited the message (e.g., added the fill price). If the text differs even slightly, it labels the message a "revision" — and on the revision path the code **never checks the bookmark**. It sends the order again. If the message keeps being re-delivered/re-pollled, it keeps sending: 17, 34, 53, 71, 75 times.

Important correction (verified Aug 5): it was NOT only false-positive text-diff revisions. In both confirmed cases the channel really DID edit the message (Telegram records an edit date) — and even those genuine edits re-opened the trade instead of updating the existing one. The bug is that the revision path re-enters, full stop: no edit, no amendment. Both a text difference and a real edit produce a duplicate order.

The duplicated trades are real: every copy has its own broker ticket number, so the broker really received N orders and N positions were really opened.

3. The Duplication Chain (Technical, with exact lines)

#	Link	Where	What it does
1	Entry settle poll	<code>worker/src/userListener.ts:1922-1934</code> — <code>scheduleEntryMessageSettlePoll()</code> ; delays from <code>ENTRY_MESSAGE_SETTLE_MS</code> (default 10s) + 30s (<code>entryMessageSettleDelaysMs()</code> , <code>userListener.ts:162-176</code>)	Re-fetches the Telegram message 10s/30s after entry
1b	Reconcile sweeps & poll hooks	listener reconcile machinery — observed in prod log as <code>source=reconcile_reconcile_sweep</code> and <code>source=reconcile_reconcile_poll_hook</code>	Re-dispatch the SAME message as a revision every few minutes (4–5 revision dispatches per signal observed in one 2h window)
1c	Catchup	<code>catchUpOnStartEnabled()</code> — <code>userListener.ts:182-190</code> ; observed as <code>source=catchup</code>	After listener reconnect/restart, catch-up re-parses recent messages → revision dispatches
1d	Live edit	Telegram edit event handler; observed as <code>source=live_edit</code>	Channel actually edits the message → revision dispatch
2	False revision	<code>userListener.ts:1963</code> — <code>storedMessageDiffersFromTelegram(...)</code> ; <code>userListener.ts:1978</code> — <code>tryApplyMessageRevision()</code>	Any text difference (settled price, formatting) → treated as a message revision, no Telegram edit-date requirement
3	Revision dispatch	<code>userListener.ts:1829</code> — <code>dispatchRow.dispatch_source = MESSAGE_REVISION_DISPATCH_SOURCE</code> ; <code>userListener.ts:1865</code> — <code>this.dispatchRevisionSignal(dispatchRow)</code>	Re-pushes the signal to the trade worker with <code>message_revision</code> source
4	Revision flag	<code>worker/src/tradeExecutor/dispatch.ts:526</code> — <code>isMessageRevision = opts?.dispatchSource === MESSAGE_REVISION_DISPATCH_SOURCE</code> ; <code>dispatch.ts:846</code> — <code>sameSignalRefresh: isMessageRevision</code>	Propagates the flag into <code>send0rder</code> options
5	Claim bypass	<code>worker/src/tradeExecutor/TradeExecutor.ts:1466-1471</code> — <code>if (!isRevisionRefresh) { ... await claimSignalBrokerDispatch(...) }</code>	The ONLY anti-duplicate guard is skipped on revisions → <code>OrderSend</code> proceeds unchecked
5b	Amend-only guard is conditional (falls through)	<code>worker/src/tradeExecutor/entryPrepare.ts:360-387</code> — revisions route to <code>tryParameterFollowUpMergeModifyOnly</code> ; <code>mergeRouting.ts:34-38</code> — no open basket → <code>{handled:true, success:false}</code> (skip); <code>mergeRouting.ts:58-59, 63-64, 87</code> — <code>!hasFxsocketConfigured()</code> / re-enter intent / no API / non-buy-sell → <code>{handled:false}</code>	When the merge path returns <code>handled:false</code> , execution falls through to the normal entry path (<code>runRangeEntry</code> / <code>runSingleEntry</code>) → the plan re-opens. This is the intended "modify only" safety net for revisions, but it only applies for manual/range-strict configurations with <code>FxSocket</code> and a resolvable anchor
6	<code>OrderSend</code>	<code>worker/src/tradeExecutor/orderLegExecution.ts:196-235</code> — <code>getFxClient().orderSend(...)</code> / <code>api.orderSend(...)</code>	Places a real broker order; max 2 attempts per leg, but no cap on whole-dispatch re-inocations

Listeners events confirm the loop: `entry_settle_poll_mismatch` → `message_revision_applied` → `message_revision_dispatch_deduped` (dedupe only caught 1 of N).

3.1 Listener-log proof (prod, Aug 4, docs/Prod_Logs/Listener/logs.1785871948065.log , 09:07–11:22 UTC)

One message, re-dispatched by multiple mechanisms. Signal `22628a24` (msg #17279, 53 orders):

```
10:47:07.511 [userListener] dispatch signal ... signalId=22628a24 messageId=17279 + original, dispatch #1
10:47:21.179 [userListener] message revision dispatch ... signalId=22628a24 source=entry_settle_poll
10:47:33.243 [userListener] message revision dispatch ... signalId=22628a24 source=catchup
10:47:56.784 [userListener] message revision dispatch ... signalId=22628a24 source=reconcile_reconcile_poll_hook
```

Revision dispatches per signal in this 2h window (all `user=dd18ad68`): `ce211b02` ×4, `b199d15e` ×4, `a5cd28c2` ×4, `5a56f595` ×4, `22628a24` ×3, `39e6d69d` ×3, `0dff3ec3` ×3 — i.e. **4–5 total dispatches per message** (1 original + 3–4 revisions). Sources observed: `entry_settle_poll` , `catchup` , `reconcile_reconcile_sweep` , `reconcile_reconcile_poll_hook` , `live_edit` . Every one of these reaches the trade worker as `message_revision` → `sameSignalRefresh` → claim bypassed → the plan executes again.

4. Evidence

4.1 Claims vs orders (the smoking gun)

Signal `906a4b64` (msg #17284, Aug 4 11:52):

```
signal_broker_dispatch_claims: 1 row (created 11:52:40.055, BEFORE the first order)
trades: 34 rows (opened 11:52:43.274 → 11:53:00.625)
metaapi_order_id: 34 DISTINCT tickets → 34 real broker orders
```

The system's own bookmark said "dispatched once" — and 33 more orders were sent anyway.

4.2 Worker log proof (signal 29d7d97f, in the captured log window)

```
14:42:19.863 claim created
14:42:21.358 OrderSend ok ticket=449551618 (first position)
14:42:29.301 sendOrder again + ~10s later = the settle-poll delay
14:42:30.304 OrderSend ok ticket=449551887 (SECOND position)
```

Zero "skip duplicate dispatch claim" / "skip duplicate in-flight" / "claim insert failed" log lines — the claim was never consulted on the second send.

4.2b Execution-log proof (signal ead1ebb8, FTMO account 8556fff2, Aug 4 13:41)

Three separate successful `order_send` actions in `trade_execution_logs` for ONE signal, identical payload (volume 0.41, comment `TScopier:44sClub:ead1ebb8`, no `:tpN` suffix — the single-entry plan, not a multi-layer plan):

13:41:33.614	<code>order_send</code>	<code>success</code>	<code>price=4088.83</code>	<code>volume=0.41</code>	<code>comment=TScopier:44sClub:ead1ebb8</code>
13:41:49.246	<code>order_send</code>	<code>success</code>	<code>price=4088.65</code>	<code>volume=0.41</code>	<code>comment=TScopier:44sClub:ead1ebb8</code>
13:41:57.181	<code>order_send</code>	<code>success</code>	<code>price=4089.87</code>	<code>volume=0.41</code>	<code>comment=TScopier:44sClub:ead1ebb8</code>

Signal `ead1ebb8` has `telegram_edit_date_seen = 1785850911` (13:41:51 UTC) — the channel really did edit the message; the edit triggered order #3. Order #2 (13:41:49) came from the settle poll. The 3 trades: identical lot 0.41 / SL 4077 / TP 4097, 3 distinct broker tickets (281762049, 281762205, 281762266). **All 3 closed together 2026-08-05 00:19:33.138.**

4.2c Both confirmed signals had REAL Telegram edits

Signal	created_at	telegram_edit_date_seen	orders
<code>906a4b64</code> (44Fx, msg #17284)	11:52:38.709	1785844360 = 11:52:40	34
<code>ead1ebb8</code> (44's Club, msg #14238)	13:41:30.350	1785850911 = 13:41:51	3

This is decisive for the fix: the duplicate re-entry was NOT only caused by false-positive text-diff revisions. The settle-poll produced order #2 without an edit, and the REAL edit produced order #3 (and #2 of the 34-group). So the revision path must never re-enter — it must amend the open basket (or skip when the basket is already in the exact target state), regardless of whether an edit date is present.

4.3 Not configuration, not multiple accounts

- All 34 rows identical: XAUUSD **sell**, lot 0.03, SL 4093.00, TP 4073.00 — one plan cloned. Layering varies lots/SL per layer; this does not.
- All 34 on **one** account (`13da4830` "MT5 Demo for 1 Chanel") — not a 14-account fan-out.
- Exceeds his own caps: `multi_trade_max_orders: 20` (TSA config 26) and `max_trades_per_zone: 3` — 34/71/75 orders violate his settings.
- Control users on the same system: 1.0–2.4 trades/signal. Luis: **19.9**; user 14bf6329: **51.8** (110 trades from one signal).
- Verified account configs (live DB, 2026-08-05): `13da4830` (MT5 Demo for 1 Chanel) = manual / multi / range_trading, config last updated **Jul 20**; `8556fff2` (FTMO USD 100K fonded) = manual / **multi** / range_trading, config last updated **Jun 22**; `9e869a6f` (ICMarketsSC-Demo) = manual / multi / range_trading. **All three are multi-style with `add_new_trades_to_existing: true`** — there is no "single" account in Luis's set. The 3 FTMO orders are still duplication (identical lot/SL/TP, no `:tpN` layer suffix, one per revision) but they are NOT evidence of "single-style account opened 3" — correct the earlier framing: the account is multi; the 3 orders are the same single-entry plan executed 3×.

4.4 Secondary bug: `channel_id = NULL` bypasses the channel filter

- Duplicated 44Fx signal rows (msgs 16xxx-17xxx) have `channel_id = NULL` in `signals` (verified for `906a4b64`) — cross-user lookup identifies the underlying channel as "44Fx".
- Account `13da4830` has `enforce_signal_channel_filter: true` with `signal_channel_ids = [TSA, Signal Tester]` — yet it traded 44Fx messages (the NULL channel evades the filter).
- NOT applicable to `ead1ebb8`** : that signal has `channel_id = 9aa18946` (44's Club), which IS in FTMO's (`8556fff2`) allowed list `[af54130c, 9aa18946]` — the FTMO group passed the channel filter legitimately; its duplication is purely the revision re-entry bug.
- 2 extra duplicate signal rows per NULL-message channel were also found (Gold Pro, TSA: `telegram_message_id IS NULL` duplicates).

5. Timeline

Date	Event
2026-07-23	First observed duplication (msg #2922 → 20 trades) — undetected, no alerting
2026-07-23 → 08-04	Continuous duplication; ~1,300+ excess trades for Luis alone
2026-08-04 17:01	Luis last signed in; complaint about duplicated trades
2026-08-04 18:37	Log captured (<code>logs.1785865057109.log</code>)
2026-08-04	Investigation: DB queries + log forensics → root cause chain identified (this doc)

6. Proposed Fix

Priority order:

Proposed fixes in plain English

- Make the one-time safety check run for every entry attempt, including message revisions.
 - When a message changes, update the existing broker trade instead of opening another trade.
 - If the system cannot safely update the existing trade, stop and log the revision. It must not fall through to a new entry.
 - Add a database safety rule so the same broker order cannot be saved twice.
 - Reject entry signals that do not have a known, permitted channel unless the account explicitly allows unknown channels.
 - Alert the team when one signal creates more than three trades on the same account within five minutes.
 - Keep a user-facing record of affected trades so compensation and support decisions can be handled accurately.
 - Extend the existing admin signal and trade drill-downs so an administrator can see the complete dispatch history, idempotency checks, and every linked broker trade in one place.
- `worker/src/tradeExecutor/TradeExecutor.ts` — **stop the bypass (root fix). Implemented locally:** the claim check now runs for revisions as well as original dispatches. A revision that encounters an existing claim is allowed to continue only into the amend-only path; it cannot be treated as permission to send a new entry. The normal duplicate-claim path still returns without execution. Unexpected claim-database errors now fail closed instead of allowing `OrderSend`.

- Verified 2026-08-05:** `entryPrepare.ts` now routes every `sameSignalRefresh` revision to `tryParameterFollowUpMergeModifyOnly` before entry-zone, teaser, range, or `OrderSend` logic. The merge router returns `handled: true, success: false` for unavailable FxSocket/API, re-enter intent, unsupported actions, or no open basket, so no revision condition falls through to a new entry.
- `worker/src/userListener.ts` — **settle-poll must not re-enter. Behavior protected by the worker fix:** the settle poll may still detect a changed Telegram message and dispatch a revision for reconciliation, but that revision is now amend-only or safely skipped. It cannot reach a new entry plan. Both confirmed signals had REAL `telegram_edit_date_seen` values, so requiring a real edit date alone remains insufficient.
 - DB defense-in-depth** — `supabase/migrations/20260805000000_trades_idempotency_guard.sql` . **Implemented locally; not applied to a database yet.** The migration first checks for historical duplicate broker tickets and stops with an audit error if any exist. If the preflight is clean, it creates a unique partial index on `trades (broker_account_id, metaapi_order_id)` and an index for signal/account trade-family audits.
 - No one-open-trade constraint is being added: legitimate multi/range/layered strategies can have multiple open legs.
 - Channel filter hardening** — `worker/src/tradeExecutor/ (dispatch/sendOrder)`. Treat `channel_id = NULL` on an entry signal as *deny* unless the broker explicitly allows unknown channels; log + skip. Closes the filter bypass that let `13da4830` trade non-allowed channels. (Does not apply to `ead1ebb8` — its channel was legitimately allowed.)
 - Alerting.** Fire when any signal produces >3 trades on a single (user, broker_account) within 5 minutes (would have caught this on Jul 23, day one).
 - Support/data (Luis).** Full per-user excess-trade report generated (56 groups / 1,408 trades). The 3 FTMO duplicates (signal `ead1ebb8`) **closed themselves together on 2026-08-05 00:19:33** — no close/keep decision needed anymore; the remaining decision is per-user compensation. Note for the support conversation: Luis's accounts are ALL multi/range style — the "single trade duplicated" framing should be "a single signal opened the same order multiple times".
 - Admin dashboard observability — extend the existing drill-downs.** The `tscopier-admin` signal and trade detail views should show, for each signal and broker account:
 - expected entry count, actual trade count, excess count, and a duplication-risk status;
 - the claim state and timestamps: claim created, claim checked, claim rejected/lost, released, or skipped;
 - every dispatch attempt with its source (`original` , `entry_settle_poll` , `catchup` , `reconcile_reconcile_sweep` , `reconcile_reconcile_poll_hook` , or `live_edit`), revision number, and timestamp;
 - the linked trades with trade ID, broker ticket (`metaapi_order_id`), symbol, direction, lot size, entry price, stop-loss, take-profit, opened/closed times, status, and order comment;
 - the execution result for each attempt, including retries, errors, `OrderSend` / `OrderModify` , and whether the action was an entry, amendment, or safe skip;
 - the Telegram message and channel details, including `channel_id` , edit date, message ID, and the reason a revision was processed;
 - a timeline that makes the relationship clear: one signal → dispatches → claims → execution attempts → broker trades.

Use the existing `SignalDetailModal` , `TradePipelineModal` , user activity tabs, `signals.pipeline_ts` , `trade_execution_logs` , `listener_events` , and `signal_broker_dispatch_claims` data where available. Add only the missing idempotency and duplication fields; do not create a parallel trade-detail experience.

7. Idempotency Analysis

Is the system idempotent today? No.

Idempotency in plain English

Idempotency means that repeating the same request does not create another real-world action. For this system, processing the same Telegram signal more than once should not open another broker position.

- The system currently remembers that a signal was dispatched, but one processing path ignores that record.
- Because that record is ignored, the same signal can place the same order several times.
- The database also allows duplicate trade records, and the broker request has no unique request key.
- The target behavior is simple: one signal and one broker account should produce one entry action. Later message changes should update that existing position, not create another one.

Layer	Guard	Status
DB <code>trades</code>	<code>id</code> PK only; <code>signal_id</code> NOT unique	✗ a signal can (and did) produce 75 rows
DB <code>trades</code>	unique (<code>broker_account_id</code> , <code>metaapi_order_id</code>) guard	✓ migration created locally; pending preflight and database application
Worker	<code>signal_broker_dispatch_claims</code> UNIQUE (<code>signal_id</code> , <code>broker_account_id</code>), checked BEFORE entry execution	✓ worker source now checks every dispatch, including revisions; revisions continue only into amend-or-skip behavior; deployment pending
Broker	<code>OrderSend</code> has no client-supplied idempotency key	✗ retry after timeout → duplicate fill possible (the 0.37s cadence shows this happening)

Design for idempotency (target state):

- The atomic claim insert (`signal_broker_dispatch_claims`) IS the idempotency key for "this signal+broker was dispatched". The worker source now checks it unconditionally on every dispatch, including revisions.
- Revisions become amend-only: claim exists + open basket → `OrderModify`; flat basket → release claim, then re-enter. The existing merge machinery (`tryParameterFollowUpMergeModifyOnly`) is the mechanism, but it must not fall through to entry when its guard conditions fail — it must skip.
- `metaapi_order_id` uniqueness at the DB makes double-persist of one broker order impossible once the migration is applied.
- Every trade row already has a stable UUID `id` (PK, `gen_random_uuid()` , verified 0 NULLs) — the row identity is fine; the problem is duplicate rows representing duplicate orders.

Admin tracking required for the target state:

- Show one signal-level record that joins the signal, broker account, claim, dispatch attempts, execution logs, and all resulting trades.
- Compare expected trades with actual trades and flag `actual > expected` , repeated broker tickets, repeated order comments, or multiple entry actions after the first claim.
- Record explicit outcome events for `execution_claim_lost` , `message_revision_dispatch_deduped` , `merge_routed_modify_only` , `revision_amended_existing_trade` , `revision_skipped_existing_trade` , and `duplicate_trade_detected` .

4. Make every alert and table row clickable into the existing signal or trade detail modal, with filters for user, broker account, signal, channel, date, status, dispatch source, and risk level.
5. Preserve the raw evidence alongside the plain-English explanation so support staff can understand the issue without losing the broker ticket and execution details needed for verification.

8. Files Involved

- `worker/src/tradeExecutor/TradeExecutor.ts` (claim skip at :1466; sendOrder guard flow :1423-1529)
- `worker/src/tradeExecutor/entryPrepare.ts` (:358-408 — revision → merge routing & fall-through to entry)
- `worker/src/tradeExecutor/basketMerge/mergeRouting.ts` (:34-38 no-open-basket skip; :58-64, :87 conditional `handled:false` → fall-through)
- `worker/src/tradeExecutor/basketMerge/sLTPRefresh.ts` (amend/refresh mechanics)
- `worker/src/tradeExecutor/dispatch.ts` (:526, :842-848 — revision flag & sendOrder invocation)
- `worker/src/userListener.ts` (:1825-1866 revision dispatch; :1922-1994 settle poll; :2637-2639 scheduling)
- `worker/src/tradeExecutor/signalBrokerDispatchClaim.ts` (claim/release helpers — logic OK, only called conditionally)
- `worker/src/tradeExecutor/revisionIdempotency.test.ts` (revision safe-skip and fail-closed claim tests)
- `worker/src/tradeExecutor/orderLegExecution.ts` (:183-335 OrderSend loop)
- `supabase/migrations/20260529120000_signal_broker_dispatch_claims.sql` (existing guard)
- `tscopier-admin/src/components/SignalDetailModal.tsx` (signal-level drill-down and execution history)
- `tscopier-admin/src/components/TradePipelineModal.tsx` (trade-level drill-down and broker execution details)
- `tscopier-admin/src/components/user/UserSignalsTab.tsx` and `UserTradesTab.tsx` (user-level filtering and navigation)
- `docs/admin-trade-type-classification.sql` (read-only SQL Editor preflight and actual execution-type classification)
- `docs/incident-2026-08-04-trade-duplication.md` (this report)

9. Follow-ups

1. **Implement fix #1 (revision path honors claim, amend-only) + #2 (settle poll never re-enters) on a branch from `upstream/dev` : Implemented locally 2026-08-05.** Worker build passed; 32 focused regression tests passed. The full worker test command only reported the first two suites before exiting, so broader test-runner coverage still needs a separate confirmation.
2. Run `docs/admin-trade-type-classification.sql` in the Supabase SQL Editor, audit any duplicate broker tickets and unknown execution types, then apply and test migration `20260805000000_trades_idempotency_guard.sql` . Do not add a one-open-trade constraint because legitimate multi/range/layered accounts can have multiple open legs.
3. Deploy the worker fix and database guard to staging, then verify: no signal with >2 trades/account within 24h; alert #5 fires on regression.
4. ~~Decide the 3 open FTMO duplicates~~ **Resolved 2026-08-05:** all 3 FTMO duplicates closed themselves (00:19:33 UTC). Remaining: per-user compensation decision.
5. Consider backfilling `channel_id` for the 16xxx/17xxx signal rows (currently NULL) to restore filter behavior.
6. Extend the existing admin dashboard drill-downs with per-signal/per-broker expected-versus-actual trade counts, actual execution type (`single` , `range` , `layered` , `range + layered` , duplicate replay candidate, or unknown), dispatch and claim timelines, linked broker tickets, execution attempts, duplication-risk filters, and the events listed in section 7. The user-detail trade modal now shows evidence-based type labels and needs live worker events and deployed claim outcomes to show the full post-fix picture.